1) What is the Big Oh:
factorial1 has a Big Oh of **O(n)**
factorial2 has a Big Oh of **O(n)**
fibonacci1 has a Big Oh of **O(n)**
fibonacci2 has a Big Oh of **O(2^n)**

2) Towers of Hanoi
E. *O(2ˆn)*

T(n) = 2 T(n−1) + 1
a= 2
b=1
fn =1
n^(log_b(a)) = n^(log_1(2)) = n^0 = 1
T(n) = O(2^n)
The worst case number of moves is O(2^n).

3) Greedy Algorithms

Whats the capacity of the knapsack in cubic inches: 4500
The suggested items are: 4 EasyBakeOvens, 27 Barbies, 1 GIJoe with a total value of $2218. There were 5 cubic inches left unused.

4) Amortized Analysis of Quicksort Algorithm

While some input arrays may lead to an O(n2) time complexity, if the pivot is uniformly chosen randomly, it has a high probability of O(n log n) time regardless of the initial array. For example, if the input is an already sorted array, only picking the first or last element the worst-case time will occur. For a large array (for example, n=100000) the probability to randomly choose the first or last element is very small (for this example, 0.00002).

5) Linear Programming
13 boxes of cookies and 9 boxes of muffins is the most you can make.

x_1 = number of **cookie boxes**
x_2 = number of **muffin boxes**

Constrains
Flour: 80 packs
Milk: 80 jugs

	Flour packs	Jugs	Profit
Cookies	4	3	$150
Muffins	3	1	$100
Available	80	80	

Balanced Solution has the greatest profit
Initially:
$150x_1 + 100x_2$
$4x_1 + 3x_2 <= 80$
$5x_1 + 1x_2 <= 80$

Isolating x_2
$4x_1+3(80-5x_1) <= 8$
$4x_1+240-15x_1<=80$
$-11x_1+240<=80$
$-11x_1<=-160$
$x_1 = 14.54$ rounded
$x_1 = 14$

$5x_1 + 1x_2 <= 80$
$5(14) + 1x_2 <= 80$
$70 + 1x_2 <= 80$
$x_2 = 10$

Then from here I just inserted number
$4(14) +3(10) =$
$56+30 = 86$ is bigger than 80 so it won’t work

Best total
$150(13) + 100(9)$
$1950 + 900 =$ **$2850**

Best solo Cookie

Flour: 4x_1 <= 80 -> 20
Milk: 5x_1 <= 80 -> 16

150 × 16 = **$2400**

Best solo Muffin
Flour: 3x_2 <= 80 -> 26.6
Milk: 1x_2 <= 80 -> 80

100 × 26 = **$2600**

6) Randomized algorithm to find an element in an array

```
import random
def alea():
    import random
    arr = []
    for _ in range(333):
        arr.append(2)
    for _ in range(333):
        arr.append(3)
    for _ in range(334):
        arr.append(4)
    random.shuffle(arr)
    return arr

def pick(a):
    index = random.randint(0, len(a) - 1)
    if a[index] == 2:
        return index
    else:
        return None

arr = alea()
index = pick(arr)
if index is not None:
    print(f"Value at index {index}: {arr[index]}")
else:
    print("index is None not found")
```

7) Advanced Data Structures
A: Express the map as an adjacency matrix.

```
graph = [
    [0, 442, 4, 0, 0, 1344, 0, 0],
    [442, 0, 0, 0, 842,932, 0, 0],
    [4, 0, 0, 437, 0,0, 0, 0],
    [0, 0, 437, 0, 0,399, 337, 0],
    [0, 842, 0, 0, 0,376, 0, 345],
    [1344,932,0, 399, 376, 0, 0, 154],
    [0, 0, 0, 337, 0, 0, 0, 221],
    [0, 0, 0, 0, 345, 154, 221, 0]
    ]
```

B: Give the length of the shortest route in lightyears.

```
=== Starting out from the Earth these are the shortest paths ===

To reach Pleiades from Earth, Invader Zim's path must be:
Earth -> Pleiades
Total distance traveled would be 442 lightyears.
Splits:
   Earth -> Pleiades is 442 lightyears
About 13 pints of Romulan Ale is required.

To reach Alpha Centauri from Earth, Invader Zim's path must be:
Earth -> Alpha Centauri
Total distance traveled would be 4 lightyears.
Splits:
   Earth -> Alpha Centauri is 4 lightyears
About 0 pints of Romulan Ale is required.

To reach Vela Molecular Ridge from Earth, Invader Zim's path
must be:
Earth -> Alpha Centauri -> Vela Molecular Ridge
Total distance traveled would be 441 lightyears.
Splits:
   Earth -> Alpha Centauri is 4 lightyears
   Alpha Centauri -> Vela Molecular Ridge is 437 lightyears
About 13 pints of Romulan Ale is required.

To reach Rosette Nebula from Earth, Invader Zim's path must be:
Earth -> Alpha Centauri -> Vela Molecular Ridge -> Orion Nebula
-> Rosette Nebula
Total distance traveled would be 1216 lightyears.
Splits:
```

```
   Earth -> Alpha Centauri is 4 lightyears
   Alpha Centauri -> Vela Molecular Ridge is 437 lightyears
   Vela Molecular Ridge -> Orion Nebula is 399 lightyears
   Orion Nebula -> Rosette Nebula is 376 lightyears
About 38 pints of Romulan Ale is required.

To reach Orion Nebula from Earth, Invader Zim's path must be:
Earth -> Alpha Centauri -> Vela Molecular Ridge -> Orion Nebula
Total distance traveled would be 840 lightyears.
Splits:
   Earth -> Alpha Centauri is 4 lightyears
   Alpha Centauri -> Vela Molecular Ridge is 437 lightyears
   Vela Molecular Ridge -> Orion Nebula is 399 lightyears
About 26 pints of Romulan Ale is required.

To reach Canis Major from Earth, Invader Zim's path must be:
Earth -> Alpha Centauri -> Vela Molecular Ridge -> Canis Major
Total distance traveled would be 778 lightyears.
Splits:
   Earth -> Alpha Centauri is 4 lightyears
   Alpha Centauri -> Vela Molecular Ridge is 437 lightyears
   Vela Molecular Ridge -> Canis Major is 337 lightyears
About 24 pints of Romulan Ale is required.
```

C: List the stops on the shortest route from Earth to Orion.

```
To reach Orion from Earth, Invader Zim's path must be:
Earth -> Alpha Centauri -> Vela Molecular Ridge -> Orion Nebula
-> Orion
Total distance traveled would be 994 lightyears.
Splits:
   Earth -> Alpha Centauri is 4 lightyears
   Alpha Centauri -> Vela Molecular Ridge is 437 lightyears
   Vela Molecular Ridge -> Orion Nebula is 399 lightyears
   Orion Nebula -> Orion is 154 lightyears
About 31 pints of Romulan Ale is required.
```

8) Approximated Algorithms
For each of the examples below, state why an approximated solution is a reasonable choice:

A: For computing the value of **π**
Since we can't know what the remaining digits are and how far they go. It took 158 days when a study was done in March of 2022, and they only reached the 100 trillionth digit and there are more left that will remain unknown. Pi is an irrational number meaning infinite and non-repeating.

B: For a machine learning algorithm based on decision trees
A machine learning algorithm is made to recognize patterns and adjust. But there are still possible external unpredictable events which are inputs that can affect the ML algorithms result so they cannot be exact, it can only be approximated with possible delineations. ML algorithms use real world information which isn't always accurate and can change. Decision trees use patterns so approximation is the best I can do.

C: For the traveling salesperson problem
There are too many variables to considered what the most efficient path would be. With each city visited the number of routes increases factorially. To compute the best routes isn't reasonable cause that expensive computing time. So, approximation is the most reasonable.

D: For a strategy game available on a mobile device
To have all the permutations of a possible decisions for a move from the mobile device to win would take a lot of time and energy and phones have limited computational resources which isn't reasonable for it to try and figure every possible move and the best one at that. So, it's better to limit them with approximation as doing all that work on a phone could limit battery life. Those chess machine that the Grandmasters play against are powerful.